MultiAgent RAG Platform for SBI

May 21, 2026

Retrieval Strategy Matches Data Type

Data Type	Retrieval Strategy
CSV / Excel	TableRAG
PDF	Hierarchical Structure-Aware RAG
Markdown	Structure-Aware RAG
Plain Text	Vector RAG
Government Schemes Database	SQL Retrieval
External Websites	Web Retrieval Agent
Voice Queries	Speech-to-RAG Pipeline

Table 1: Document-type-specific retrieval strategies.

Metadata-First Architecture

Every document, chunk, embedding, and retrieval operation preserves rich metadata. Metadata quality often contributes more to retrieval accuracy than embedding model selection, especially when users ask questions scoped by file, state, language, section, or source.

Goal 1: Canonical Government Schemes Data Platform

Objective

Create a normalized repository of government scheme information stored in SQLite with APIs and frontend access.

Methodology

Government scheme datasets often contain inconsistent field names, duplicated records, and varying structures. The ingestion layer normalizes column names, state names, scheme identifiers, ministry names, URLs, and dates into a canonical schema.

Listing 1: Canonical scheme fields.

```
scheme_id
scheme_name
state
ministry
department
eligibility
benefits
application_process
required_documents
official_website
source_url
last_updated
```

SQLite indexes should cover `scheme_name`, `state`, and `ministry`. The API exposes list, filter, and detail endpoints.

Listing 2: Scheme API endpoints.

```
GET /schemes
GET /schemes?state=Karnataka
GET /schemes?name=PM Kisan
GET /schemes/{id}
```

The frontend displays canonical fields, official links, related schemes, source documents, and retrieval sources. Success means sub-second lookup, a consistent schema, and reliable SQL retrieval.

Goal 2: Intelligent Upload and Strategy Selection

Objective

Allow users to upload files and automatically determine the best embedding and retrieval strategy. Supported types include CSV, Excel, PDF, Markdown, Word, HTML, plain text, and audio.

File Type	Strategy
CSV	TableRAG
Excel	TableRAG
PDF	Hierarchical RAG
Markdown	Structure-Aware RAG
Text	Vector RAG
Audio	Speech Pipeline

Table 2: Upload routing logic.

Success requires no manual configuration, correct strategy selection, and support for mixed document collections.

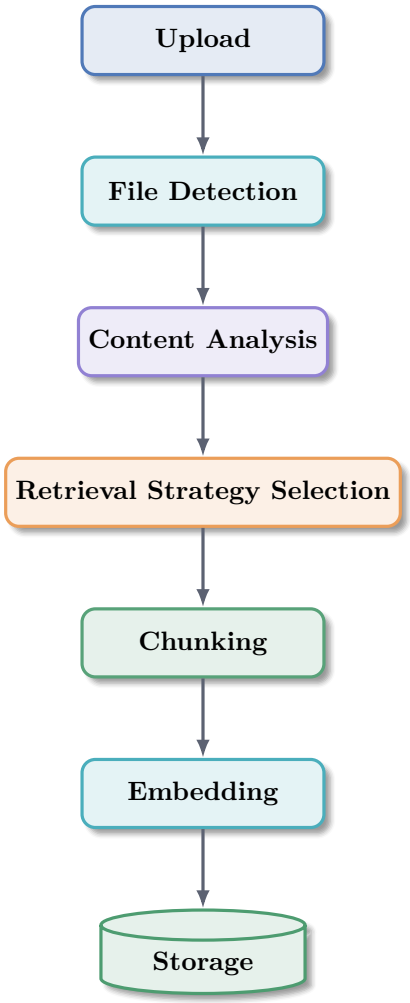

Figure 1: Automatic file processing pipeline.

Goal 3: ChromaDB Vector Store and Embedding Infrastructure

Objective

Create a centralized embedding platform supporting multiple retrieval methods. The embedding layer uses Nomic Embed Text V2 MOE through Ollama.

Listing 3: ChromaDB collections.

```
table_documents
pdf_documents
markdown_documents
text_documents
audio_transcripts
```

Required features include incremental indexing, batch indexing, re-indexing, metadata filtering, and multi-collection search.

Goal 4: Metadata-First Storage Architecture

Objective

Store rich metadata alongside every embedding so retrieval can be filtered by file, state, language, source, section, and strategy.

Listing 4: Example metadata record.

```
{
  "document_id": "uuid",
  "filename": "pm_kisan_guidelines.pdf",
  "original_filename": "PM-Kisan-Guidelines-2025.pdf",
  "document_type": "pdf",
  "retrieval_strategy": "hierarchical_rag",
  "language": "en",
  "state": "Karnataka",
  "section": "Eligibility",
  "subsection": "Income Criteria",
  "page_number": 7,
  "chunk_id": 45,
  "parent_chunk_id": 12,
  "source": "upload",
  "upload_date": "2026-05-21"
}
```

Every chunk must store:

- `filename`, `document_id`, `document_type`, and `section`
- `language`, `chunk_id`, `upload_date`, and `retrieval_strategy`

Goal 5: Document Registry Service

Objective

Track uploaded documents independently of vector storage to support management, re-indexing, version tracking, deletion, and analytics.

Listing 5: SQLite document registry.

```
CREATE TABLE documents (
 document_id TEXT PRIMARY KEY,
 filename TEXT,
 document_type TEXT,
 retrieval_strategy TEXT,
 language TEXT,
 upload_time TIMESTAMP,
 chunk_count INTEGER,
 embedding_model TEXT,
 collection_name TEXT
);
```

Goal 6: TableRAG for CSV and Excel Files

Objective

Improve retrieval accuracy for structured tabular data, where traditional vector search can lose table structure.

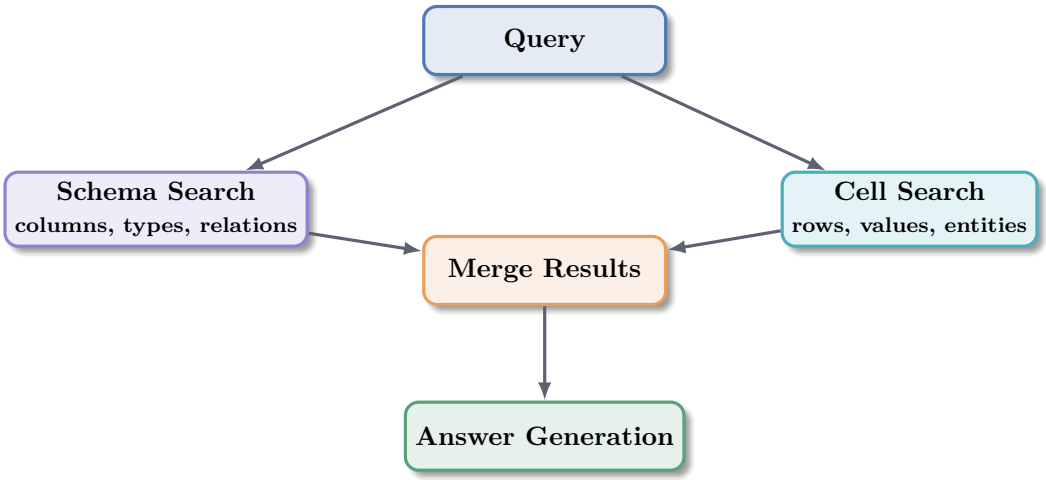

Figure 2: TableRAG combines schema and value indexes.

The schema index stores column names, descriptions, data types, and relationships. The value index stores actual rows and cell values such as `PM Kisan`, `Karnataka`, `Rs. 6000`, and `Small Farmers`.

Goal 7: Hierarchical Structure-Aware RAG

Objective

Preserve document hierarchy during indexing for PDFs and Markdown documents.

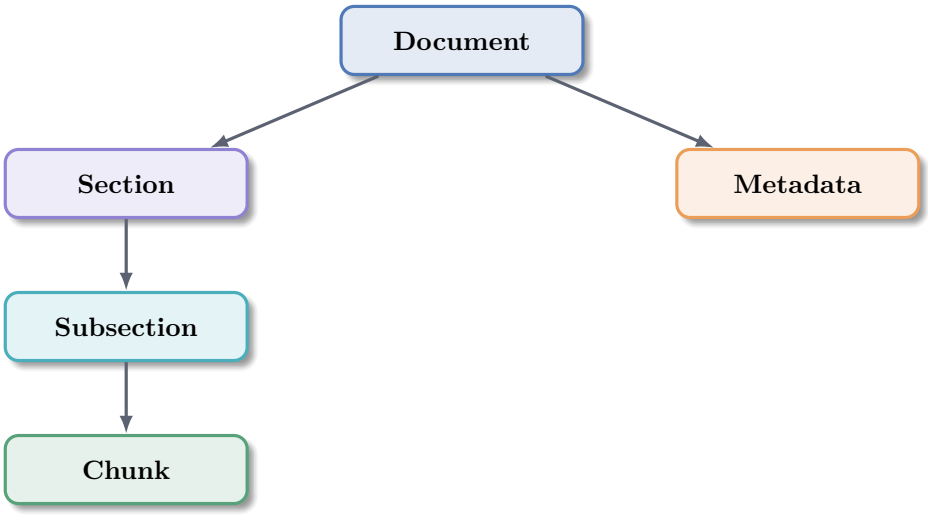

Figure 3: Hierarchical index structure for PDF and Markdown retrieval.

The extraction layer identifies titles, chapters, sections, subsections, tables, lists, and references. Retrieval proceeds from section to subsection to chunk before answer generation.

Goal 9: Agentic Retrieval Architecture

Objective

Use specialized agents instead of a single monolithic retrieval chain.

Agent	Responsibilities	Main Tools
SQLite Agent	Query schemes database, generate SQL, aggregate results, return structured answers.	SQLite, schema explorer, SQL generator
Vector Retrieval Agent	Run ChromaDB search, query expansion, metadata filtering, similarity search, reranking, and context assembly.	ChromaDB, embeddings, reranker
Document Routing Agent	Select which documents should be searched before vector retrieval.	Registry, metadata filters
Web Enrichment Agent	Retrieve additional context when local data is insufficient.	Official portals, site extraction, verification
Retrieval Evaluation Agent	Evaluate coverage, confidence, source agreement, and completeness.	Scorers, policy rules
Coordinator Agent	Receive query, determine intent, invoke agents, merge results, generate final answer.	Agent orchestration

Table 3: Specialized retrieval agents.

Vector Query Expansion Example

Input query: `PM Kisan Karnataka eligibility`. Expanded query terms include `PM Kisan`, `Pradhan Mantri Kisan Samman Nidhi`, `Farmer Income Support`, and `Eligibility Karnataka`.

Listing 6: Example vector retrieval result.

```
{
  "filename": "pm_kisan_guidelines.pdf",
  "section": "Eligibility",
  "score": 0.94
}
```

Goal 10: Hybrid Retrieval Layer

Objective

Combine SQL retrieval, vector search, TableRAG, metadata search, BM25 keyword search, and web retrieval.

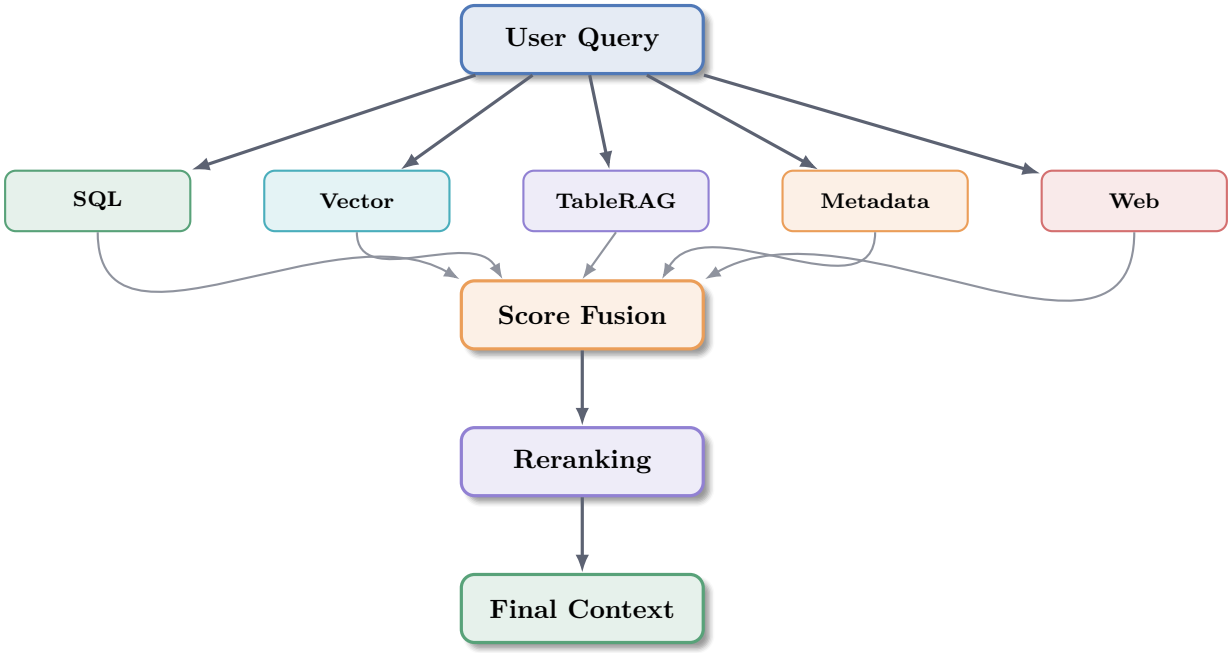

Figure 4: Hybrid retrieval and fusion layer.

Goal 11: Reranking and Context Optimization

Objective

Improve retrieval precision before generation.

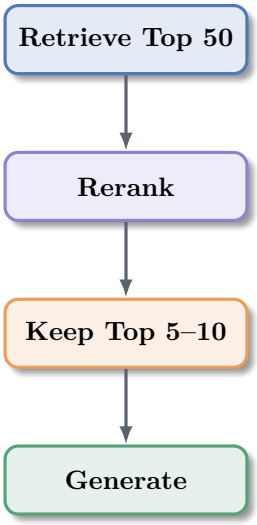

Figure 5: Context optimization pipeline.

Core features include duplicate removal, distractor detection, relevance scoring, and metadata weighting.

Complete System Architecture

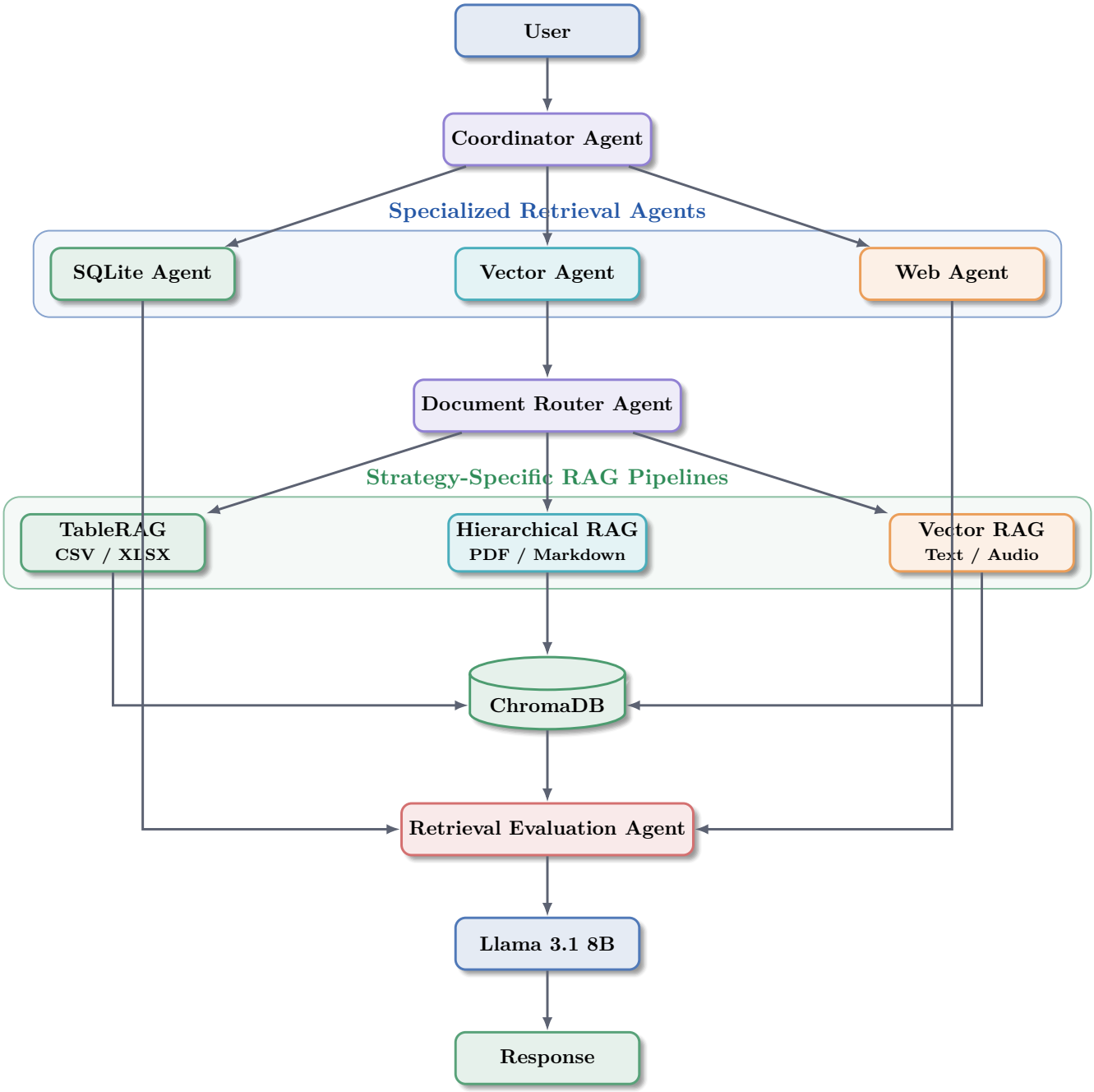

Figure 6: End-to-end agentic multimodal RAG architecture.

Implementation Milestones

1. Normalize the government schemes database and expose the initial SQLite-backed API.
2. Build the document registry and metadata validation rules.
3. Implement upload detection, document routing, and strategy selection.

4. Add TableRAG, hierarchical RAG, and vector RAG indexing pipelines.
5. Integrate ChromaDB collections, metadata filtering, and multi-collection search.
6. Add agent orchestration, retrieval evaluation, reranking, and web enrichment.
7. Ship frontend views for schemes, uploaded documents, sources, and final answers.